ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Hajer Mastouri**
- **Chinese Name:**
- **Email: hejermastouri04@gmail.com**
- **Student ID: A15922202**
- **University: NTU**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. 594 categories
2. Isolating specific categories like sci.med is crucial before running data mining algorithms because it helps when it comes to domain-specific goals so that the mined patterns align with analytical objectives as in medical literature in this case. It also reduces noise so that the meaningful patterns specific to this domain don't get pruned out as noise which might happen when all categories remain combined.

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

Dropping 40% of our data is a very naïve and flawed way to deal with missed data because it results in massive information loss, an information that is crucial to extract patterns and draw conclusions. It introduces a significant bias. So, it is not the optimal approach given data scarcity and how difficult it is to acquire high-quality text data.

An objective alternative is to separate data into two sets: one training set and one testing set. The 60% available data is the one we are going to train using a classification model to predict the missing category_name labels of the remaining 40%.

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. X_sample is a 1000-document random sample from the total data consisting of 2257 documents. The category proportions do not exactly match. In fact, the rank order has even changed (from soc.religion.christian having the most documents to comp.graphics becoming the larger category in the sample)
2. A simple random sampling can result in underrepresenting the highly imbalanced dataset. It can even totally miss a whole category that is rare in the original data and thus fails to learn relatively rare patterns because it is purely based on chance.

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. The total number of features increases from 35.482 to 35.788 (306 columns are added) when stop_words=None. That's because common English words are not filtered during tokenization. So, those words are added to the dictionary making the matrix's column dimension bigger.
2. Adding stopwords back decreases the overall sparsity from 99.6772% to 99.547% and that's because stop words appear repeatedly across almost every document. Those added columns when stop_words=None contain non-zero counts in almost every row which makes the matrix denser and reduces its proportion of zeros.

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	Number of terms kept (keep == True)
0.0	35,482
0.01	5,089
0.05	1,491

1. **As the threshold increases the number of terms kept decreases.**
2. **The words that were removed with lowest variance are words that rarely appear across all documents: rare nouns, number or even typographical error. Since these words have a count of zero in almost all rows their values do not vary resulting in near-zero variance.**
3. **Filtering low-variance terms wholly reduces dimensionality without losing significant power. It helps in reducing memory and training time for models. It also prevents overfitting and memorizing noise. A high threshold however might eliminate terms that have moderate frequency yet define well a certain category.**

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

3. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
4. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
5. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

3. The top-correlated words are a mix of both, but domain-specific terminology seems to be more present under Spearman rank correlation than under Pearson correlation. For example, for comp.graphics, Pearson correlation is heavily-influenced by generic conversational words such as 'thanks', 'writes', 'looking', 'hi' and the core 'graphics' word is ranked much lower. However, in Spearman correlation, domain-specific words like 'graphics', 'files', '3d', 'image' are much more prioritized. This correlation also identifies words like 'people' or 'god' which are not so related to the terminology for graphics.
4. Pearson correlation measures the strength and direction of a strictly linear relationship between raw numerical values. It assumes continuous variables and is sensitive to the actual magnitude of values, making it susceptible to outliers and extreme word counts.
   Spearman correlation measures the strength and direction of a monotonic relationship by computing the Pearson correlation on the ranked transformations of the data rather than raw counts
   It evaluates whether an increase in one variable corresponds to an increase in the other, regardless of whether that relationship is linear or non-linear. Because ranks bound the impact of scale, Spearman is non-parametric and resistant to skewness and outliers.
5. It depends on the consistency and whether the word reliably shows up when the topic is discussed. An example of that is when a word appears 1 or more times across all posts in comp.graphics but is absent in all other categories, this produces a high Spearman correlation. However, if the word appears in few documents many times like 80, the extreme outliers produce a severe non-linear variance which results in lower Pearson correlation. So the scenario occurs when a word appears consistently but with extreme count outliers.

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup:

Decreasing minSup from 9 to 3 results in an increase in the number of discovered patters (from 710 to 9,547) : lowering the frequency threshold allows many low frequency token combinations to meet the criteria for being considered frequent. and it also results in an increase in execution runtime (from 0.35 to 9.17 secs) : candidate pruning becomes less aggressive, the conditional FP-trees grow much deeper as the threshold drops.

Answer (2) — Top-K k vs MaxFPGrowth minSup:

Parameter K enforces an exact output size. The algorithm returns exactly the top k patterns with highest support in the dataset without making the user guess the threshold in advance. However, minSup enforces a fixed frequency : it extracts all itemsets that satisfy the condition support >= minSup so the final number of retrieved patterns remains unpredictable (threshold set too high can yield 0 patterns and one that is slightly too low can result in a combinatorial explosion).

Answer (3) — Maximal vs standard frequent patterns:

A maximal frequent pattern is when none of its supersets are frequent. In standard mining whenever a larger pattern is frequent, every single subset of it is also listed which results in redundancy. Maximal pattern mining only retains comprehensive pattern eliminating the intermediate subcombinations thus getting rid of all redundancies.

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. A standard visual scatter plot exists in only 2 dimensions or 3. In contrast, the raw X_counts matrix contains over 35000 features where every unique vocabulary word acts as a distinct coordinate axis. Mathematically and perceptually, human visual systems cannot comprehend or perceive a point in 35000-dimensional space without projecting it down to 2 or 3 components.
2. Dimensionality reduction must be applied first. To reduce the high-dimensional feature space down to 2 components while preserving variance, techniques such as PCA Principal Component Analysis or t-SNE t-Distributed Stochastic Neighbor Embedding can be used.

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method (**PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. T-SNE and UMAP produce much more distinct and separated clusters with clear boundaries whereas PCA shows overlap around the center.
2. PCA is a linear projection technique and it seeks orthogonal axes to maximize global variance across the whole dataset. In high dimensional text spaces (+35000 dimensions), there are complex non-linear relationships that govern class boundaries and not simple linear hyperplanes. So compressing this high-dimensional space onto 2 linear principal components results in overlapping classes. However, the other two learning methods are non-linear and do prioritize preserving local neighborhood relationships rather than global Euclidean distances : They pull documents with similar vocabulary close together.
3. They must avoid overinterpreting cluster distances because t-SNE makes optimizations for local structure rather than global distances. So distances between separated clusters are arbitrary and non-interpretable.

AI used for this answer: Yes / No

If yes, how:

External references (links):

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. 4 binary columns are created to represent 4 categories. For every vector, one position contains 1 indicating the class while the others remain 0.
2. It assumes an ordinal ranking, an order of categories based on the numbers associated to each position when in fact all categories are discrete and unordered classes.
3. One-hot binarization projects each category onto its own orthogonal coordinate axis in a higher-dimensional space. No false ranking is imposed on the machine learning model since all pairwise distances between different classes are identical.

AI used for this answer: Yes / No

If yes, how:

External references (links):
